Recursion, Tree Recursion

Computer Science Mentors 61A

September 21, 2026 - September 25, 2026

1 Recursion Basics

1. What are three parts you find in every recursive function?

Created by Ana Abell, Tien-Yu Hsu, Cadence Loh, and Rajoshi Basu. feedback form!
Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

2. What is wrong with the following function? How can we fix it?

```
def factorial(n):
    return n * factorial(n)
```

3. Complete the definition for `sum_prime_digits`, which returns the sum of all the prime digits of `n`. Recall that 1 is not prime. Assume you have access to a function `is_prime`; `is_prime(n)` returns `True` if `n` is prime, and `False` otherwise.

```
def sum_prime_digits(n):
    """
    >>> sum_prime_digits(12345)
    10 # 2 + 3 + 5
    >>> sum_prime_digits(4681029)
    2 # 2 is the only prime number
    """
    if _____________________________________________:

        return _____________________________________

    if _____________________________________________:

        return _____________________________________

    return _____________________________________
```

2 Writing Recursive Functions

1. Complete the definition for `num_digits`, which takes in a number n and returns the number of digits it has . Implement the function recursively.

```
def num_digits(n):
    """Takes in an positive integer and returns the number of
    digits.

    >>> num_digits(0)
    1
    >>> num_digits(1)
    1
    >>> num_digits(7)
    1
    >>> num_digits(1093)
    4
    """
```

2. (Spring 2015 MT1 Q3C) Implement the `combine` function, which takes a non-negative integer `n`, a two-argument function `f`, and a number `result`. It applies `f` to the first digit of `n` and the result of combining the rest of the digits of `n` by repeatedly applying `f` (see the doctests). If `n` has no digits (because it is zero), `combine` returns `result`.

```
def combine(n, f, result):
    """
    Combine the digits in non-negative integer n using f.

    >>> combine(3, mul, 2) # mul(3, 2)
    6
    >>> combine(43, mul, 2) # mul(4, mul(3, 2))
    24
    >>> combine(6502, add, 3) # add(6, add(5, add(0, add(2, 3))))
    16
    >>> combine(239, pow, 0) # pow(2, pow(3, pow(9, 0))))
    8
    """
    if n == 0:
        return result
    else:
        return combine(_______ , _______ ,

                       ________________________________)
```

3 Recursive Helper Functions

1. Suppose the **position** of the rightmost digit of a number is defined as 1. The position of a digit increases from right to left.

   (a) Fill in the `positionizer` function below. It takes in a non-negative integer `n` and returns a non-negative integer. For each digit `d` of `n`, `positionizer` either changes `d` to be the remainder of `d` divided by its position, or leaves `d` the same if it is equal to its position.

```
def positionizer(n):
    """
    >>> positionizer(12)
    10
    >>> positionizer(23)
    20
    >>> positionizer(12345)
    12300
    """
    def helper(n, pos):

        if _____________________________________________:

            return _____________________________________

        rest = _____________________________________
        if n % 10 == pos:

            return rest + _____________________________
        else:

            return rest + _____________________________

    return helper(_________________, _________________)
```

(b) Now fill in the `max_positionizer` function below. Given a list of digits, it returns the `k`-digit number with the highest value when positionized. The digits must stay in the same order. (ex: for `k = 2` and `lst = [1,2,3]`, you can't form 21). You may use `positionizer` in your solution.

Hint: Use the helper to create a list of all possible k digit numbers formed from `lst`.

```
def max_positionizer(k, lst):
  """
  >>> max_positionizer(2, [1, 2, 3]) # positionized version
                                     # of 12, 13, 23 are
                                     # 10, 10, 20
  23
  >>> max_positionizer(3, [2, 5, 3, 1])
  251
  """
  def make_nums(k,lst):

    if ____________________________________:

      return ____________________________________

    elif ____________________________________:
      return []

    a =  [____________________________________ \

              for rest in ____________________________________]

    b = ____________________________________________
    return a + b

  return ________(make_nums(_____, _____), _____________)
```

4 Tree Recursion

1. What is the difference between recursion and tree recursion? Give an example of a problem that might require tree recursion.

2. The *Gibonacci sequence* is a recursively defined sequence of integers; we denote the nth Gibonacci number g_n. The first three terms of the sequence are $g_0 = 0, g_1 = 1, g_2 = 2$. For $n \geq 3$, g_n is defined as the sum of the previous three terms in the sequence.

   Complete the function `gib`, which takes in an integer `n` and returns the nth Gibonacci number, g_n. Also, identify the three parts of recursive function design as they are used in your solution.

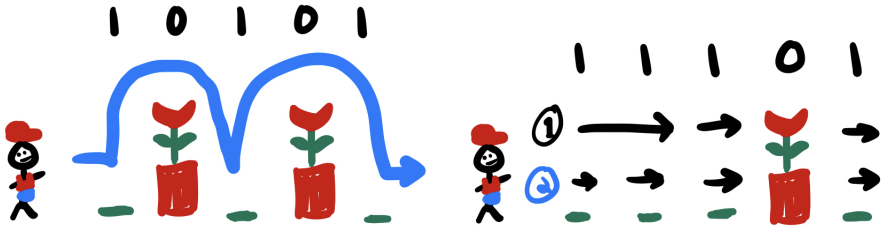
```
def gib(n):
    """
    >>> gib(0)
    0
    >>> gib(1)
    1
    >>> gib(2)
    2
    >>> gib(3) # gib(2) + gib(1) + gib(0) = 3
    3
    >>> gib(4) # gib(3) + gib(2) + gib(1) = 6
    6
    """
    if _____________________________________:

        return _____________________________________

    return _____________________________________
```

3. Including the original call, how many calls are made to `gib` when you evaluate `gib(5)`?

4. Mario needs to get from one end of a level to the other, but there are deadly Piranha plants in his way! Mario only moves forward and can either *step* (move forward one space) or *jump* (move forward two spaces) from each position. A level is represented as a series of ones and zeros, with zeros denoting the location of Piranha plants. Mario can step on ones but not on zeros. How many different ways can Mario traverse a level without stepping or jumping into a Piranha plant? Assume that every level begins with a 1 (where Mario starts) and ends with a 1 (where Mario must end up).



Hint: Does it matter whether Mario goes from left to right or right to left? Which one is easier to check?

```
def mario_number(level):
    """
    >>> mario_number(10101)
    1
    >>> mario_number(11101)
    2
    >>> mario_number(100101)
    0
    """
    if ____________________________:

        ____________________________

    elif ____________________________:

        ____________________________

    else:

        ____________________________________________________________
```

5. Implement the function `make_change`, which takes in a non-negative integer amount in cents `n` and returns the minimum number of coins needed to make change for `n` using 1-cent, 3-cent, and 4-cent coins.

```
def make_change(n):
    """
    >>> make_change(5) # 5 = 4 + 1 (not 3 + 1 + 1)
    2
    >>> make_change(6) # 6 = 3 + 3 (not 4 + 1 + 1)
    2
    """

    if ____________________________:
        return 0

    elif ____________________________:

        ____________________________________________

    elif ____________________________:

        ____________________________________________
    else:

        ____________________________________________
```